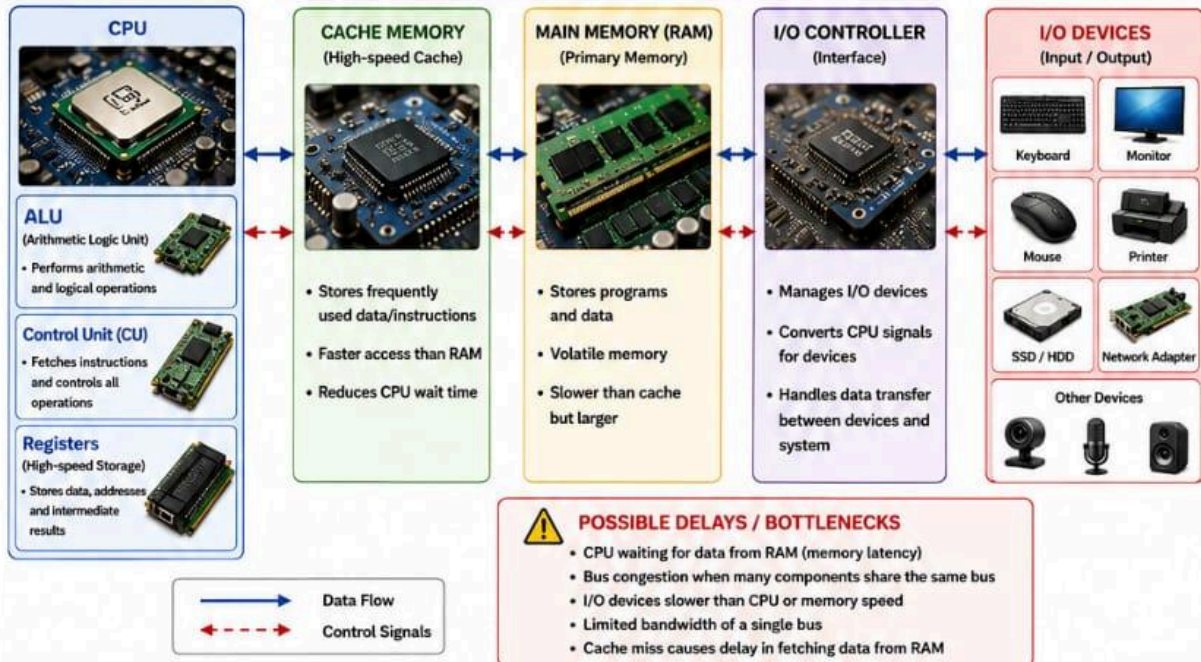
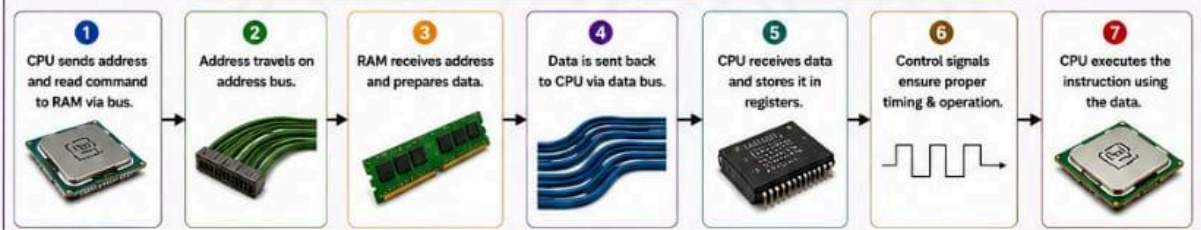


# COMPUTER SYSTEM ARCHITECTURE & BUS STRUCTURES

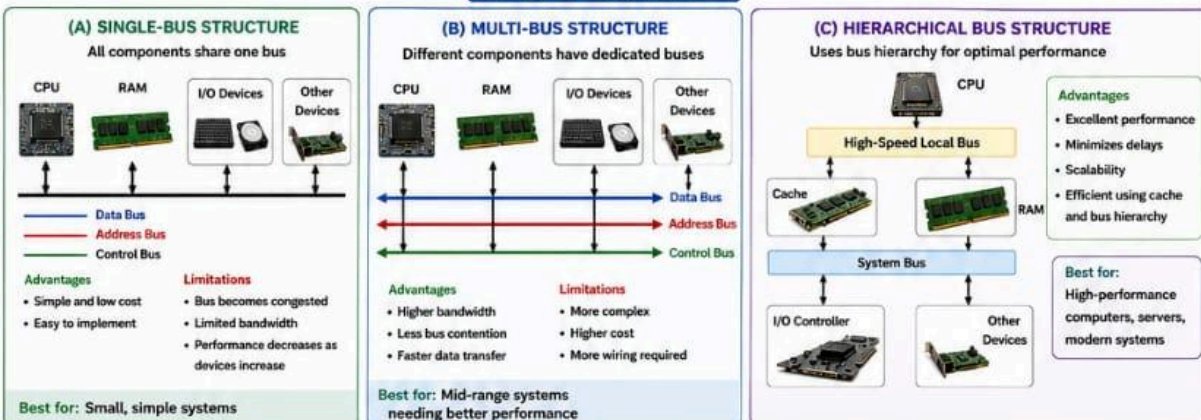
## 1. CORE COMPONENTS & DATA / CONTROL FLOW



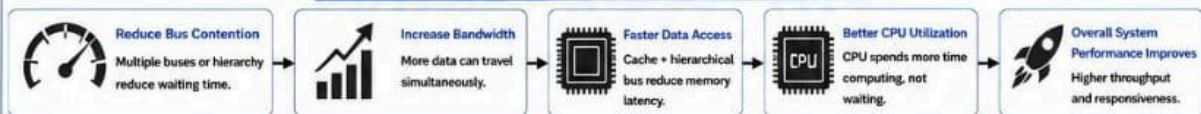
## DATA & CONTROL FLOW DURING EXECUTION (EXAMPLE: CPU READS DATA FROM RAM)



## 2. BUS STRUCTURES



## 3. HOW BUS STRUCTURES IMPROVE PERFORMANCE



### KEY TAKEAWAY

Bus structure plays a critical role in system performance. From simple single-bus to advanced hierarchical buses, the goal is to move data faster, reduce delays, and keep the CPU busy.



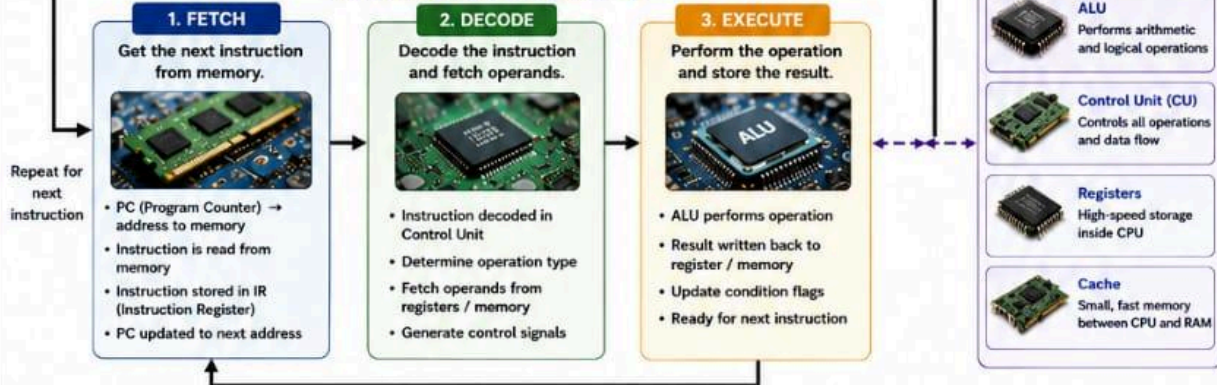
# INSTRUCTION EXECUTION CYCLE & CPU PERFORMANCE

## Concept Map

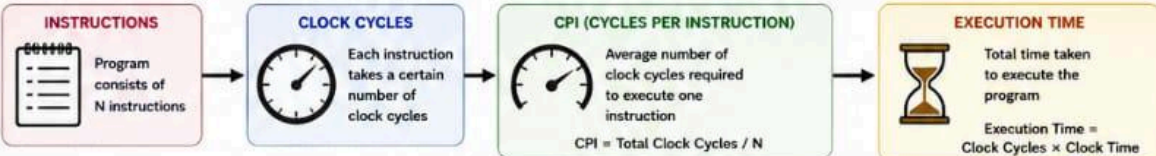
### OVERVIEW

A program consists of a sequence of instructions. The CPU repeatedly goes through the Instruction Execution Cycle to complete each instruction.

### INSTRUCTION EXECUTION CYCLE



### FROM INSTRUCTION CYCLE TO PERFORMANCE



### KEY PERFORMANCE EQUATIONS

$$\text{Total Clock Cycles} = N \times CPI$$

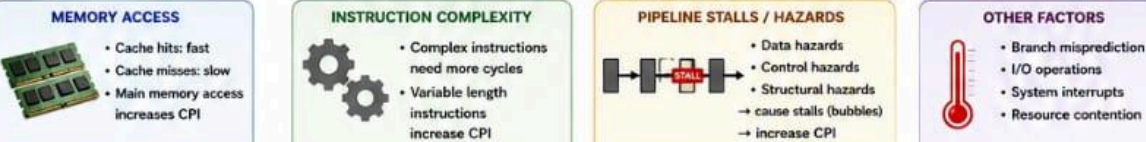
$$\text{Execution Time} = N \times CPI \times \text{Clock Time}$$

$$\text{Clock Time} = 1 / \text{Clock Frequency}$$

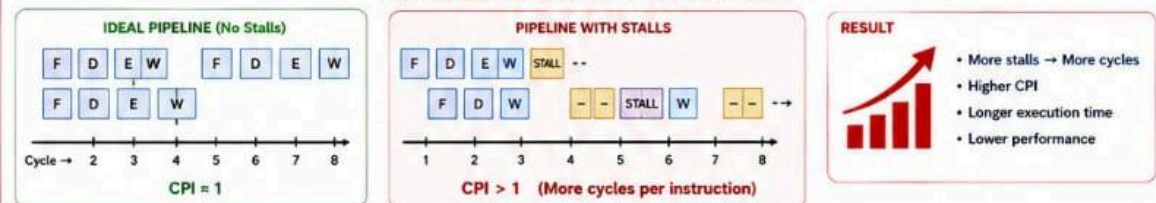
$$\text{Performance} \propto 1 / \text{Execution Time}$$

(Lower time = Higher performance)

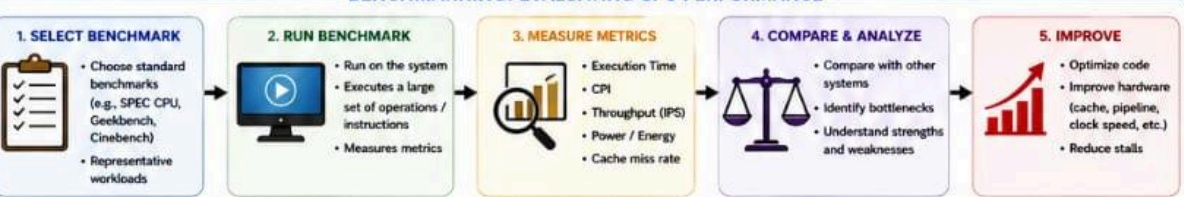
### FACTORS THAT AFFECT CPI AND EXECUTION TIME



### HOW PIPELINE STALLS IMPACT PERFORMANCE



### BENCHMARKING: EVALUATING CPU PERFORMANCE

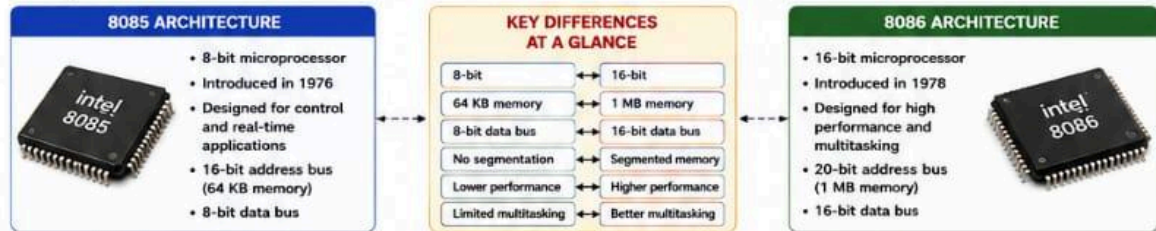


### SUMMARY FLOW

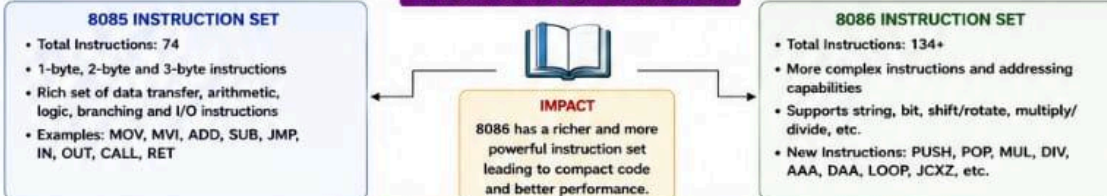


# 8085 vs 8086 ARCHITECTURE – CONCEPT MAP

## Two Generations of Intel Microprocessors



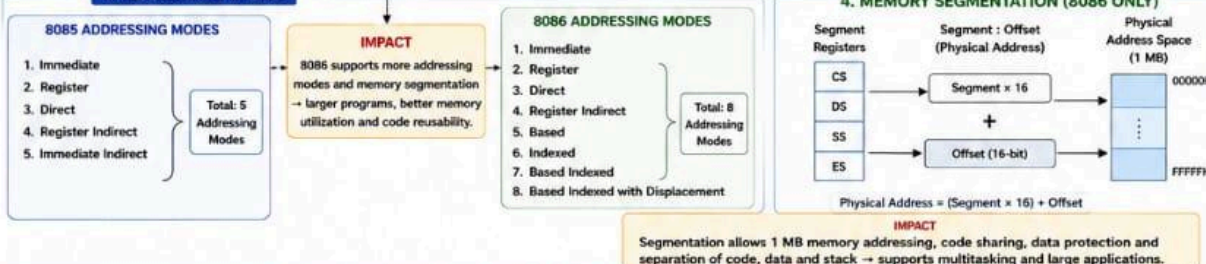
## 1. INSTRUCTION SET COMPARISON



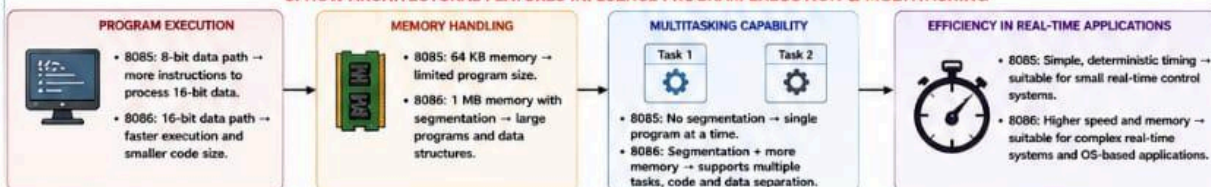
## 2. REGISTER ORGANIZATION COMPARISON



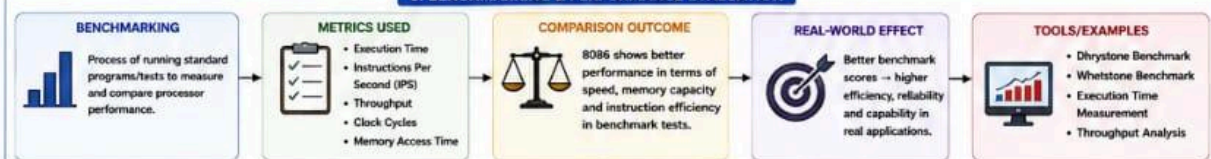
## 3. ADDRESSING MODES



## 5. HOW ARCHITECTURAL FEATURES INFLUENCE PROGRAM EXECUTION & MULTITASKING



## 6. BENCHMARKING & PERFORMANCE EVALUATION



## 7. SUMMARY COMPARISON TABLE

| Feature             | 8085                                  | 8086                                           | Impact on Performance & Applications                    |
|---------------------|---------------------------------------|------------------------------------------------|---------------------------------------------------------|
| Architecture        | 8-bit                                 | 16-bit                                         | 8086 handles more data per cycle → faster processing    |
| Address Bus         | 16-bit (64 KB)                        | 20-bit (1 MB)                                  | 8086 supports larger memory → large programs            |
| Data Bus            | 8-bit                                 | 16-bit                                         | Higher data transfer rate in 8086                       |
| Registers           | 6 GP (8-bit), SP, PC                  | 8 GP (16-bit), 4 Segment, BP, SI, DI, IP       | 8086 provides more flexibility and memory handling      |
| Addressing Modes    | 5                                     | 8                                              | 8086 more addressing options → efficient coding         |
| Instruction Set     | 74 Instructions                       | 134+ Instructions                              | 8086 richer instruction set → compact and powerful code |
| Memory Segmentation | No                                    | Yes (CS, DS, SS, ES)                           | 8086 enables multitasking and memory protection         |
| Multitasking        | Limited                               | Better                                         | 8086 supports OS and multitasking environments          |
| Best Use            | Control systems, small real-time apps | Complex applications, OS, multitasking systems | 8086 more suitable for modern computing needs           |

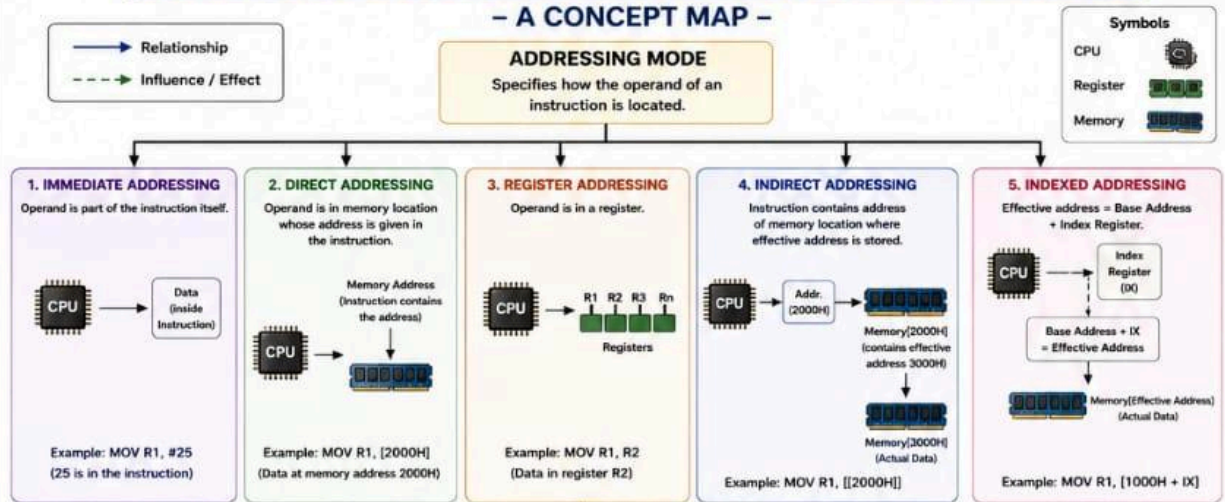


### CONCLUSION

8085 was a milestone 8-bit microprocessor ideal for simple control and real-time tasks, while 8086 introduced 16-bit architecture, segmentation and advanced features that paved the way for modern, high-performance and multitasking computing systems.

# ADDRESSING MODES, INSTRUCTION TYPES & THEIR IMPACT

## - A CONCEPT MAP -



## HOW ADDRESSING MODES CONNECT TO INSTRUCTION TYPES

| INSTRUCTION TYPES<br>Categories of instructions in a processor.                                                                  | ADDRESSING MODES APPLICABLE |                                |                           |                                  |                                   |
|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------|--------------------------------|---------------------------|----------------------------------|-----------------------------------|
|                                                                                                                                  | Immediate (#)               | Direct (address)               | Register (R)              | Indirect (@ / [ ])               | Indexed (base + IX)               |
| <b>A. DATA TRANSFER INSTRUCTIONS</b><br>Move data between registers, memory and I/O.<br>Examples: MOV, LDA, STA, XCHG, PUSH, POP | Yes<br>(e.g., MOV R1, #25)  | Yes<br>(e.g., MOV R1, [2000H]) | Yes<br>(e.g., MOV R1, R2) | Yes<br>(e.g., MOV R1, [[2000H]]) | Yes<br>(e.g., MOV R1, [1000H+IX]) |
| <b>B. ARITHMETIC INSTRUCTIONS</b><br>Perform mathematical operations.<br>Examples: ADD, SUB, MUL, DIV, INC, DEC                  | Yes<br>(e.g., ADD R1, #10)  | Yes<br>(e.g., ADD R1, [2000H]) | Yes<br>(e.g., ADD R1, R2) | Yes<br>(e.g., ADD R1, [[2000H]]) | Yes<br>(e.g., ADD R1, [1000H+IX]) |
| <b>C. CONTROL INSTRUCTIONS</b><br>Change the sequence of program execution.<br>Examples: JMP, JZ, CALL, RET, LOOP                | Yes<br>(e.g., JMP #1234H)   | Yes<br>(e.g., JMP 3000H)       | Yes<br>(e.g., JMP R1)     | Yes<br>(e.g., JMP [3000H])       | Yes<br>(e.g., JMP [1000H+IX])     |

## EFFECT OF ADDRESSING MODES ON PERFORMANCE FACTORS

| Addressing Mode                                               | Memory Access Time (Relative)                             | Instruction Size (Relative) | Execution Speed (Relative) | Why?                                                               |
|---------------------------------------------------------------|-----------------------------------------------------------|-----------------------------|----------------------------|--------------------------------------------------------------------|
| <b>Immediate (#)</b><br>Data inside instruction               | Very Low<br>(No memory access)                            | Small                       | Fastest                    | No extra memory read needed. Operand is in the instruction itself. |
| <b>Direct (address)</b><br>Direct memory address              | Medium<br>(1 memory access)                               | Medium                      | Medium                     | Requires one memory access to fetch the operand.                   |
| <b>Register (R)</b><br>Operand in register                    | Very Low<br>(No memory access)                            | Small                       | Very Fast                  | Operands are in CPU registers. No memory access.                   |
| <b>Indirect (@ / [ ])</b><br>Address in memory points to data | High<br>(2 memory accesses)                               | Large                       | Slow                       | First read effective address, then fetch actual data.              |
| <b>Indexed (base + IX)</b><br>Base address + index            | Medium to High<br>(1 extra calculation + 1 memory access) | Medium to Large             | Medium                     | Extra time for address calculation, then memory access.            |

## REAL-WORLD SCENARIOS - WHERE EACH MODE IS PREFERRED

|                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IMMEDIATE ADDRESSING</b><br><b>Preferred When:</b> <ul style="list-style-type: none"> <li>Constant values are used</li> <li>No memory access desired</li> <li>Quick initialization</li> </ul> <b>Real-World Examples:</b> <ul style="list-style-type: none"> <li>Loading constants e.g., MOV R1, #100</li> <li>Setting I/O port values</li> <li>Configuration settings</li> </ul> | <b>DIRECT ADDRESSING</b><br><b>Preferred When:</b> <ul style="list-style-type: none"> <li>Accessing global variables</li> <li>Fixed memory locations</li> <li>Simple programs</li> </ul> <b>Real-World Examples:</b> <ul style="list-style-type: none"> <li>Accessing sensor data from fixed address</li> <li>Reading/writing to memory-mapped peripherals</li> </ul> | <b>REGISTER ADDRESSING</b><br><b>Preferred When:</b> <ul style="list-style-type: none"> <li>High-speed operations</li> <li>Frequent data manipulation</li> <li>Loop and arithmetic heavy code</li> </ul> <b>Real-World Examples:</b> <ul style="list-style-type: none"> <li>Calculations in control systems</li> <li>Signal processing</li> <li>Data movement between CPU registers</li> </ul> | <b>INDIRECT ADDRESSING</b><br><b>Preferred When:</b> <ul style="list-style-type: none"> <li>Pointers and dynamic data</li> <li>Linked lists, stacks, queues</li> <li>Large memory structures</li> </ul> <b>Real-World Examples:</b> <ul style="list-style-type: none"> <li>Operating system structures</li> <li>Dynamic memory allocation</li> <li>Complex data structures</li> </ul> | <b>INDEXED ADDRESSING</b><br><b>Preferred When:</b> <ul style="list-style-type: none"> <li>Array or table processing</li> <li>Efficient traversal of data</li> <li>Base + offset addressing</li> </ul> <b>Real-World Examples:</b> <ul style="list-style-type: none"> <li>Accessing array elements</li> <li>Look-up tables</li> <li>Buffer processing in communication systems</li> </ul> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## HOW IT ALL INFLUENCES PROGRAM EXECUTION



## KEY TAKEAWAY

- Register and Immediate modes provide highest speed.
- Direct mode is simple and widely used.
- Indirect mode is flexible but slower due to multiple memory accesses.
- Indexed mode is ideal for arrays, tables and real-time data processing.
- Choosing the right addressing mode improves performance, reduces memory usage and is critical in real-time & embedded systems.

# NUMBER SYSTEMS, INSTRUCTION REPRESENTATION & DEBUGGING – CONCEPT MAP

**OVERVIEW:** Computers use binary internally. We use hexadecimal as a compact, human-readable form.

## 1. NUMBER SYSTEMS

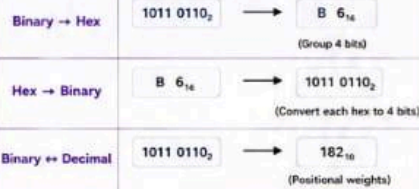
**BINARY (Base 2)**  
Digits: 0, 1  
Used inside CPU, memory and digital circuits.

Example:  
 $10110110_2$

**HEXADECIMAL (Base 16)**  
Digits: 0-9, A-F  
Each hex digit = 4 bits (one nibble).

Example:  
 $B6_{16}$

## 2. CONVERSION BETWEEN FORMATS

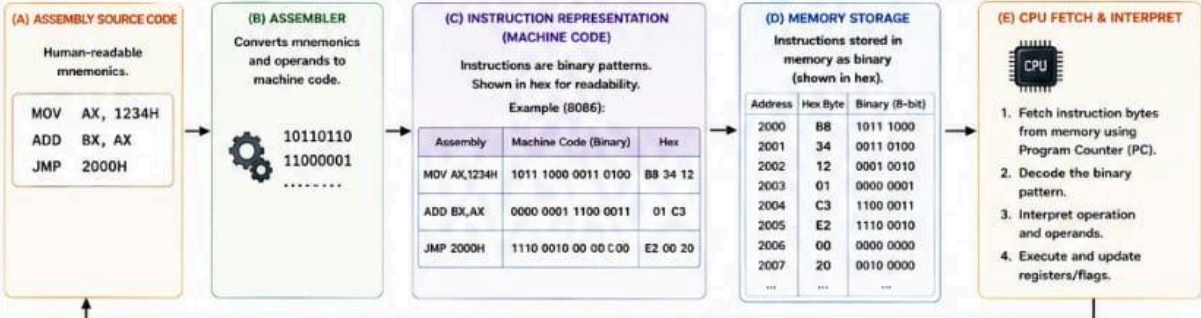


## 3. WHY HEXADECIMAL?

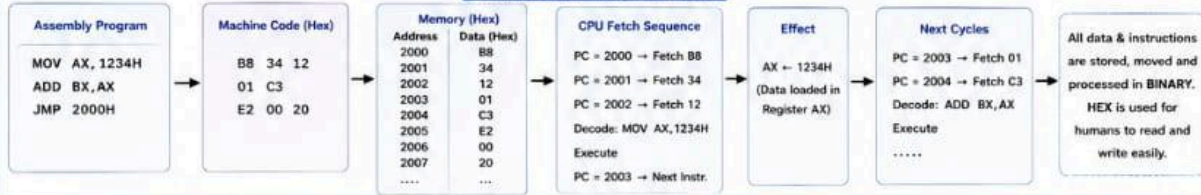
- ✓ More compact: 4 bits = 1 hex digit
- ✓ Easy to read/write for humans
- ✓ Matches memory organization
- ✓ Widely used in assembly, C, debuggers, memory dumps, networking, and hardware docs.

Example:  
 $1111\ 1111_2 = FF_{16} = 255_{10}$

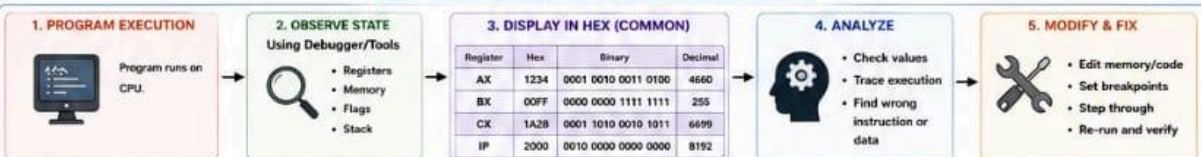
## 4. FROM NUMBERS TO INSTRUCTIONS – HOW IT WORKS



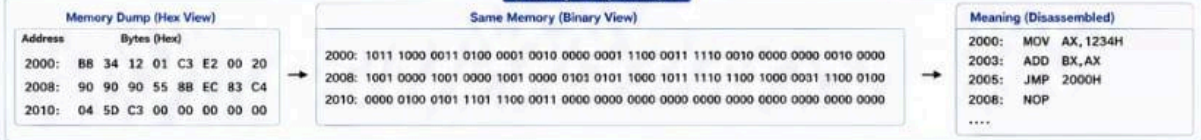
## EXAMPLE WALKTHROUGH (8086)



## 5. DEBUGGING PROCESS – ROLE OF NUMBER SYSTEMS



## Memory Dump Example



## 6. WHEN TO USE WHICH ADDRESSING MODE? (QUICK VIEW)

| Addressing Mode | Best Used For             | Reason                   | Example          |
|-----------------|---------------------------|--------------------------|------------------|
| Immediate       | Constants, initialization | Fast, no memory read     | MOV AL, 25H      |
| Direct          | Global/static variables   | Simple address           | MOV AL, [2000H]  |
| Register        | Frequent operations       | Fastest access           | ADD AX, BX       |
| Indirect        | Pointers, dynamic data    | Access data via address  | MOV AL, [BX]     |
| Indexed         | Arrays, tables, buffers   | Base + Index calculation | MOV AL, [SI+10H] |

## 7. REAL-WORLD USE CASES

|                     |                                                    |
|---------------------|----------------------------------------------------|
| Embedded Systems    | Hex used in firmware, registers, memory maps.      |
| Operating Systems   | Memory management, context switching, page tables. |
| Device Drivers      | Hardware registers accessed via hex addresses.     |
| Network Protocols   | Packets, headers represented in hex.               |
| Reverse Engineering | Disassemblers/debuggers show hex code.             |

## 8. KEY TAKEAWAYS

- ✓ CPU understands only binary.
- ✓ Hex is a compact, human-friendly representation.
- ✓ Instructions are stored in memory as binary; shown as hex.
- ✓ Debugging relies on hex to inspect and modify system state efficiently.
- ✓ Understanding conversions and formats is essential for low-level programming, system design and debugging.

## COMPLETE FLOW – FROM HUMAN CODE TO EXECUTION AND BACK



💡 "Binary makes it work. Hex makes it human."